Spec-per-Stage: Refactoring the Interaction Interface of AI Programming—An Attention-Economics Perspective

Yang Zikun
Independent Researcher

2026 年 9 月

目录

1 引言 **6**
1.1 被忽视的另一半：AI 编程的交互界面 6
1.2 核心论点：通用规范是 AI 编程的 SQL，注意力是关键变量 6
1.3 主要贡献 6

2 背景 **7**
2.1 声明式接口：数据独立性的启示 7
2.2 规格驱动开发：先例与缺口 7
2.3 LLM 注意力的实证特征 7
2.4 阶段门禁与缺陷经济学 8

3 自由提示词的失效模式：注意力去哪儿了 **8**
3.1 结构性重复：每个会话都在重新发明语法 8
3.2 注意力稀释：不变量挤占变量 8
3.3 错误晚发现：跨阶段放大 9
3.4 形式化：规范语句与提示词方差 9
3.5 小结 9

4 SpS 范式：把提示词变成语句 **9**
4.1 核心思想：三个机制 9
4.2 规范语句的参考语法 10
4.3 规范控制台：语句、事务与历史 11
4.4 语句分解与拼装：与 F2F 的衔接 12
4.5 SQL 类比的系统映射 12

5 性质与讨论 **13**
5.1 性质一：注意力集中性 13
5.2 性质二：错误早绑定与传播有界 13
5.3 性质三：评审友好性 13
5.4 与自由提示词工作流的边界 13
5.5 成本与局限 14

6 生命周期分工模型 **14**
6.1 两个相位 14
6.2 相位迁移判据 14
6.3 交接协议 15
6.4 与“取代论”的分别 15

7 扩展讨论：开放问题 **15**

8 结论 **16**

摘要

AI 编程正在成为软件生产的主导方式，但现有讨论大多聚焦于“模型能生成什么”，而较少审视“人如何向模型下达指令”。本文从注意力经济学的视角，提出一种针对 AI 编程交互界面的重构——Spec-per-Stage（SpS，阶段规范范式）：以一套通用规范充当 AI 编程的“SQL 语法”，将一次开发任务表达为若干条规范语句；每条语句由不变的模板骨架 τ 与开发者填写的变量细节 δ 构成；语句在提交生成前经过一道轻量校验（如同 SQL 的语法与类型检查），每个阶段（产品方案设计 → 技术方案设计 → 产品实现）的出口设置人工评审门禁（如同事务提交）。

本文的三重论证是：第一，注意力是 AI 编程中真正稀缺的资源——模型的上下文注意力随无关标记的注入而稀释，人的评审注意力受带宽约束，而自由提示词把两者的注意力大量消耗在“每次重新决定项目结构”这类不变量上；第二，不变量吸收——通用规范将结构性决策从每次会话的随机变量变为确定性常量，模型与人的注意力因此重新集中于真正携带信息的变量细节，正如数据独立性曾把查询者从访问路径中解放出来；第三，门禁有界——阶段门禁使缺陷在引入后的最近边界被捕获，修复动作退化为“重新生成一条语句”，错误传播与修改半径被结构性压低。

SpS 组织 AI 编程的输入端（指令），与作者先前提出的 Feature-per-File（F2F）范式——组织输出端（文件）——互补衔接：实现语句以 F2F 特征文件为输出单元，复杂任务通过语句分解为多个独立文件的增量拼装。本文同时给出规范语句的参考语法（reference grammar）与规范控制台的交互语义（语句、校验、提交、回滚与历史），并以“活动趋势查询”为运行示例展示完整工作流；最后提出生命周期分工模型：结构生成期（0→1）使用 SpS 快速构建结构完整、功能可用的全新项目，局部微调期（1→1.1）依据相位迁移判据回落到传统 AI 编程工具，规范产物作为两相之间的交接资产。

关键词： AI 编程；注意力经济学；通用规范；声明式界面；规范语句；阶段门禁；生命周期分工；规格驱动开发

Abstract

AI programming is becoming the dominant mode of software production, yet most discussions focus on what models can generate rather than on how humans instruct them. From an attention-economics perspective, this paper proposes a refactoring of the interaction interface of AI programming—Spec-per-Stage (SpS): a universal specification grammar plays the role that SQL plays for data access; a development task is expressed as a set of specification statements, each composed of an invariant template skeleton τ and developer-supplied variable details δ; a lightweight validation pass runs before generation (as SQL syntax and type checking does), and a human review gate stands at the exit of every stage—product design, technical design, and implementation (as a transaction commit does).

Three arguments are offered. First, attention is the truly scarce resource in AI programming: the context attention of models dilutes as irrelevant tokens accumulate, the review attention of humans is bandwidth-bound, and free-form prompting spends much of both on re-deciding project structure—an invariant. Second, invariant absorption: universal specifications turn structural decisions from per-session random variables into deterministic constants, so the attention of both model and human re-concentrates on the variable details that actually carry information, just as data independence once freed query writers from access paths.

Third, gate boundedness: stage gates capture defects at the nearest boundary after their introduction, reducing repair to regenerating a single statement, which structurally suppresses error propagation and the modification radius.

SpS organizes the input side of AI programming (instructions) and complements the author's earlier Feature-per-File (F2F) paradigm, which organizes the output side (files): implementation statements emit F2F feature files, and complex projects grow by statement decomposition and incremental assembly of independent files.

The paper also gives a reference grammar for specification statements, specifies the interaction semantics of a specification console (statements, validation, commit, rollback, history), and demonstrates the full workflow on a running example.

It closes with a lifecycle division-of-labor model: the construction phase ($0 \to 1$) uses SpS to build structurally complete, functional new projects; the fine-tuning phase ($1 \to 1.1$) falls back to conventional AI tools under a phase-transition criterion, with committed specification artifacts serving as the handoff assets between the two phases.

Keywords: AI programming; attention economics; universal specification; declarative interface; specification statement; stage gate; lifecycle division of labor; spec-driven development

1 引言

本章陈述问题域与两条失控证据，给出以注意力为关键变量的核心论点，并列出本文的主要贡献。

1.1 被忽视的另一半：AI 编程的交互界面

AI 编程工具的竞争几乎全部集中在生成端：更强的模型、更长的上下文、更自主的智能体[1, 2, 3]。而交互端——人如何向模型下达指令——仍停留在“自由文本提示 + 补丁式 diff”的原始形态：开发者在对话框里描述意图，模型改写代码，开发者阅读 diff 判断对错。

两条证据表明这个形态正在失控。其一，GitClear 对大量 AI 辅助仓库的度量显示，代码重复率上升、重构减少、代码质量指标下滑 [4]——无结构的生成在无结构的指令下持续熵增。其二，开发者的实际痛感并不在“生成得不够多”，而在“改不动、看不过来、错了不知道错在哪”：这三者都不是生成能力问题，而是注意力问题——模型的上下文注意力花在了哪里，人的评审注意力落在了哪里。

现有工具对这一问题的回答是“上下文工程”：让模型看到更多。本文主张相反方向的杠杆：不是让注意力看到更多，而是让人与模型把注意力花得更少、更准。稀缺资源的浪费不在模型能力，而在交互界面的组织方式。

1.2 核心论点：通用规范是 AI 编程的 SQL，注意力是关键变量

关系模型的历史为本文提供了一个精确的类比。在 SQL 之前，数据访问是过程性的：使用者必须写出导航路径——索引、游标、循环——语言的大部分注意力消耗在“怎么取”上。Codd 的数据独立性 [5] 与 SEQUEL[6] 把访问路径交给系统，语法固定为少数子句，使用者的全部语言注意力集中于“要什么”。SQL 语法因此成为数据访问的“通用提示词”：它不随任务而变，变化的只是语句中携带的业务细节。

本文的论点是：AI 编程正处于“过程性导航”阶段。开发者用自由文本反复声明项目结构、技术栈、分层方式、风格约定——这些是“怎么建”的访问路径；而真正稀缺的资源——模型的上下文注意力 [7, 8] 与人的评审注意力——被这些不变量大量占用。与 SQL 同构的解法是：一套通用规范（specification grammar）充当 AI 编程的 SQL 语法，开发者的注意力只花在语句的变量部分——那些真正区分本项目与其他项目的细节上。

注意力是这个范式的关键变数。范式优劣的判据不是生成代码的行数，而是单位有效产出所消耗的模型注意力与人的注意力。自由提示词是一场注意力拍卖：每次会话都重新竞价结构性决策；SpS 的目标是把这些决策从拍卖台上撤下来，变成语法的常量。

1.3 主要贡献

1. 提出并论证“注意力是 AI 编程的稀缺资源”：从模型端（上下文注意力的稀释效应）与人端（评审注意力的带宽约束）给出双重证据，指出自由提示词的浪费主要发生在不变量上；

2. 提出 SpS 范式：通用规范（语法层）、轻量校验（解析层）、阶段门禁（事务层）三个机制，给出规范语句的形式化定义与参考语法（reference grammar）；

3. 给出 SQL 类比的系统映射与规范控制台（specification console）的交互语义——语句、校验、提交、回滚与历史；

4. 论证 SpS 与 F2F[9] 的互补性：SpS 组织输入端（指令），F2F 组织输出端（文件）；复杂任务通过语句分解为多个独立特征文件的增量拼装，错误传播与修改半径在两端同时被有界化；

5. 分析范式的成本与任务边界：面向正向构建，不面向探索式原型与深度重构；

6. 提出生命周期分工模型：0→1 结构生成期使用 SpS，1→1.1 局部微调期依相位迁移判据回落到传统 AI 编程工具，规范产物作为交接资产，结构性变更触发重入。

2 背景

本章沿四条线索梳理知识基础：声明式接口的历史回报、规格驱动实践的现状与缺口、LLM 注意力的实证特征，以及阶段门禁与缺陷经济学的经典结论。

2.1 声明式接口：数据独立性的启示

Codd 的关系模型 [5] 把数据访问分离为逻辑意图与物理路径两层，查询优化器负责后者；SEQUEL[6] 进一步把意图的表达固定为一种近乎自然语言的语法。Parnas 的信息隐藏判据 [10]——把易变的决策封装为模块的秘密——是同一思想在模块层的表述。本文把这一判据搬到提示词层：通用规范把“不易变的决策”封装为语法（模板骨架与常量层），把“易变的决策”留作槽位。声明式的回报在两个时代同构：SQL 时代是优化器接管访问路径，SpS 时代是模型接管结构决策——前提是结构决策必须先被语法确定下来。

2.2 规格驱动开发：先例与缺口

规格化文档有长期标准传统：IEEE 29148[11] 规定了需求文档应当写什么。近年的规格驱动工具（GitHub Spec Kit[12]、AWS Kiro[13]）把规格作为生成的中间产物，验证了“先规格后代码”路线的可行性；反向方向——从自然语言到结构化查询的语义解析 [14]——则从数据侧印证了“结构化语法优于自由文本”这一判断。但现有实践存在三个缺口：其一，规格未被当作有语法的“语句”——可校验、可分解、可提交、可回滚；其二，注意力未被当作显式的设计变量——模板的取舍缺乏统一的经济学判据；其三，阶段之间缺少事务语义——评审通过与否对流程状态的影响没有被形式化。SpS 对这三个缺口给出统一回答。

2.3 LLM 注意力的实证特征

Transformer 的注意力机制 [7] 是有限资源：上下文越长，模型对关键信息的利用越不均匀。“Lost in the Middle”效应 [8] 表明，无关标记的注入会稀释模型对相关信息的位置召回与任务准确率；提示工程综述 [15] 系统化了“如何写提示”，但将模板视为提示技巧，而非语法。对本文的推论有二：上下文中每一个含混或无关的标记都在消耗预算；结构性决策若以含混方式反复出现，其成本不止于长度，更在于方差——每次会话对同一结构问题给出不同答案，下游

产物随之漂移。人的注意力服从同样的经济学：评审者面对无结构的指令史与不限大小的 diff，注意力没有落点，评审质量随体量衰减。

2.4 阶段门禁与缺陷经济学

Royce 的阶段化模型 [16] 与 Boehm 的缺陷成本曲线 [17] 给出经典结论：缺陷发现得越晚，修复成本越高。Naur 的“编程即理论构建”[18] 提醒：程序的本质是开发者头脑中的理论，而非代码文本本身；把人完全移出回路的自主智能体 [19] 恰恰切断了理论的在场。SpS 的立场是折中的：把人放回少数关键节点（阶段门禁），同时用结构化产物与模板化槽位，把“人在场”的注意力成本压到最低——人评审的不是自由文本，而是结构化文档中的变量槽。

3 自由提示词的失效模式：注意力去哪儿了

本章把自由提示词工作流的失效归结为三种模式——结构性重复、注意力稀释、错误晚发现——并以规范语句的形式化收束。

3.1 结构性重复：每个会话都在重新发明语法

在自由提示词下，一类信息被反复声明：项目结构、技术栈版本、目录布局、命名约定、接口风格、错误处理策略。它们或者每次会话重新描述，或者被复制进越来越长的“规则文件”（如全局系统提示、仓库规则文件）。这是人类时代习惯在提示词层的延续——人类开发者只对代码执行 DRY，对指令从不执行。

F2F[9] 曾论证“复用经济学的反转”：当生成成本趋零，成本的主导项从“写出来”转移到“注意力耗掉”。提示词层的结构性重复正是该反转在输入端的表现：重复的不是代码，而是注意力。每次会话对同一批结构问题的重新声明与重新求解，都是对模型注意力与人的阅读注意力的重复征税。

3.2 注意力稀释：不变量挤占变量

把一条自由提示词的信息分解为三类：不变量 I（结构性决策，跨任务不变）、变量 V（任务细节，跨任务变化）、噪声 N（含混、冗余与自相矛盾的表述）。现有工作流的典型比例是 I 与 N 挤占 V。

模型端的机制是明确的：长而含混的提示迫使模型在生成时同时求解两类问题——“该遵循什么结构”与“该实现什么细节”。前者本是常量，却以自然语言的方差形式出现；上下文中无关与含混标记的积累进一步稀释对关键细节的召回 [8]。结果是生成分布的质量重心偏移：结构性决策方差大，细节准确率下降。

人端的机制同样明确：评审者面对的是一段对话史与一组 diff，而非结构化产物。他必须先在头脑中重建“模型被要求了什么”，才能判断“模型做了什么”。评审注意力的带宽 [18, 20] 被重建过程占用，真正用于判断细节的份额缩小。复杂度没有被消除，只是被转移给了注意力最昂贵的环节——人。

3.3 错误晚发现：跨阶段放大

软件开发天然分三个阶段：产品方案设计（做什么、给谁做）、技术方案设计（怎么组织、用什么）、产品实现（写成代码）。自由提示词工作流把三个阶段压扁成一场连续对话：意图的偏差、方案的偏差与实现的偏差在代码阶段才一起显形，且 LLM 以近乎相同的成本与置信度实现一个错误的设计 [2, 3]——错误一旦进入实现阶段，修复半径由代码组织方式决定（F2F[9] 已论证分层组织会放大修改半径）。

自由提示词的问题在更上游：它让错误更晚才被看见。没有阶段产物，就没有阶段边界；没有阶段边界，就没有“在此处捕获”的位置。缺陷成本曲线 [17] 在 AI 时代不仅成立，而且被生成速度放大：人类时代错误实现的代价是工时，AI 时代的代价是“在错误地基上高速叠加更多错误实现”。

3.4 形式化：规范语句与提示词方差

定义 1（通用规范）通用规范是一个三元组 (T, G, V)。T 为模板集，每个模板 τ 定义阶段类型与槽位集合；G 为项目常量层，记录跨语句共享的结构决策（技术栈、架构风格、命名与风格约定、质量约束）；V 为校验规则集，定义语句与产物必须满足的静态约束。

定义 2（规范语句）一条规范语句是二元组 $s = (\tau, \delta)$，其中 $\tau \in T$ 为模板骨架（不变的语法），δ 为开发者对 τ 所定义槽位的赋值（变量的细节）。语句是 AI 编程的最小工作单元。

定义 3（结构决策熵）对任务类 C，设 $S(c)$ 为完成任务 c 所需的结构决策集（架构、组织方式、风格选择）。在自由提示词 p 下，模型对 S 的后验分布随措辞变化，$H(S \mid p) > 0$；在模板 τ 下，S 的取值由 τ 与 G 完全确定，$H(S \mid \tau) = 0$。

命题 1（不变量吸收）将自由提示词替换为规范语句后，模型与人的注意力预算中分配给结构决策 S 的份额趋于下界，边际注意力重新集中于 δ；生成分布的方差相应地从结构层转移到细节层，即生成质量的不确定性只存在于开发者真正需要决策的地方。

依据 $H(S \mid \tau) = 0$ 消除了结构层的全部含混；长上下文稀释效应 [8] 意味着消除无关与含混标记对细节召回的改善是超线性的；人端评审对象从“对话史 + diff”变为“槽位赋值表”，重建成本（3.2 节）归零。

3.5 小结

自由提示词的三种失效——结构性重复、注意力稀释、错误晚发现——共享同一根源：把不变量当变量处理。第 4 节给出的范式用一个动作回应：把不变量吸收进语法，让注意力只花在变量上。

4 SpS 范式：把提示词变成语句

本章给出 SpS 范式的三个机制、规范语句的参考语法、规范控制台的交互语义、与 F2F 的衔接方式，以及 SQL 类比的系统映射。

4.1 核心思想：三个机制

SpS 由三个机制构成，分别对应语法的三个层次：

机制一：通用规范（语法层）。一套覆盖三个标准阶段的模板：τ_P（产品方案设计）、τ_D（技术方案设计）、τ_I（产品实现）。模板定义固定骨架与槽位：τ_P 的槽位如目标用户、问题陈述、范围边界、成功标准、非目标；τ_D 的槽位如架构风格、数据访问方式、接口契约、异常策略；τ_I 的槽位如输出单元、依赖语句、验收清单。项目常量层 G 承载跨语句不变量：技术栈与版本、架构风格（如 F2F）、命名约定。模板通过快捷方式实例化——如同在查询工具中键入片段模板，开发者敲入 /pd、/td、/impl 即可生成对应语句的骨架，注意力直接进入填槽。

机制二：轻量校验（解析层）。语句在提交生成前经过一道静态校验：槽位完整性（必填槽位非空）、一致性（δ 与常量层 G 无冲突）、契约闭合（语句引用的上游产物与接口均存在且可解析）。校验失败即拒绝生成——如同 SQL 解析器在执行前拒绝语法错误。这一机制的成本极低（纯静态检查），却把最廉价的一类失败从“生成后人工发现”前移到“生成前机器发现”。

机制三：阶段门禁（事务层）。每个阶段的出口设置人工评审门禁：产物通过评审，则提交（commit）进入下一阶段；否则回滚（regenerate），重新生成该阶段的语句产物。阶段门禁给予流程事务语义：任何时刻，项目状态都由“已提交的阶段产物”唯一确定，未提交的生成结果不进入项目历史。

三个机制的分工是清晰的：通用规范消除结构方差（命题 1），轻量校验前移廉价失败，阶段门禁有界化错误传播（5.2 节）。

4.2 规范语句的参考语法

模板与快捷方式是通用规范在人机界面上的形态；本节给出其背后的抽象语法——规范语句的参考语法（reference grammar）。它不是一门待实现的语言，而是语句形态的非规范性（non-normative）定义：任何工具，只要其指令形态满足该语法，即为 SpS 兼容——正如 SQL 的前端既可以是控制台，也可以是可视化查询构建器，语法的有效性不依赖于具体界面。

表 1: 规范语句的参考语法（EBNF，非规范性）

```
<console>    ::= <session> { <statement> }
<session>    ::= ".use" "spec:" <version> "project:" <identifier>
<statement>  ::= <directive> <assignment>+
<directive>  ::= "/pd" | "/td" | "/impl"
<assignment> ::= <slot> ":" <payload>
<slot>       ::= <identifier>
<payload>    ::= <value> { ";" <value> }
```

其中，⟨session⟩ 载入通用规范版本与项目常量层 G；⟨directive⟩ 对应三类阶段模板 τ_P、τ_D、τ_I；合法 ⟨slot⟩ 集合由所选模板定义；⟨payload⟩ 即开发者填写的变量细节 δ。

语法的设计原则只有一条：自由度被刻意压缩到唯一的位置。directive 与 slot 的组合由模板约束——语法承担不变量；开发者唯一自由书写的区域是 payload——注意力载荷。这与 SQL 严格同构：SELECT…FROM…WHERE 的子句骨架是常量，变化的只是字面量与绑定变量；骨架的固定不是限制，而是把使用者的全部注意力推向值的手段。4.5 节的映射表将这一同构展开为逐项对照。

轻量校验（机制二）在语法上的落点由此明确：槽位完整性即必填 ⟨assignment⟩ 非空；一致性即 ⟨payload⟩ 与常量层 G 无冲突；契约闭合即语句引用的上游产物均处于已提交状态。校验是语法的静态语义，而非附加的工艺。

4.3 规范控制台：语句、事务与历史

SpS 的工作界面是规范控制台（specification console），其交互语义直接类比 SQL 控制面板：语句是工作单元，会话历史是语句序列，提交/回滚是事务动作，项目状态是已提交产物的集合。控制台会话的一个片段如下（沿用 F2F[9] 的活动趋势查询作为运行示例，通用规范版本 v0.3）：

表 2: 规范控制台会话示例（spec v0.3）

```
> .use spec:v0.3 project:activity-board   # 载入通用规范与常量层 G

> /pd   # 实例化产品方案设计模板
PD-001 (τ_P: 产品方案设计)
    目标用户: 运营团队
    问题: 需要查看近 30 日用户活动趋势
    范围: 单看板页; 只读; 无导出
    成功标准: 打开即见趋势; P95 查询 < 1s
    非目标: 权限细分; 移动端
    [validate] 槽位 5/5 完整; 与 G 无冲突 -> pass
    [gate] 人工评审 -> commit (2026-09-17)

> /td   # 技术方案设计, 消费 PD-001
TD-001 (τ_D: 技术方案设计)
    架构风格: F2F 特征文件 [20]
    数据访问: SQL, 只读从库
    接口契约: GET /api/trend?range=30d -> TrendResponse
    [validate] 契约可解析; 上游引用闭合 -> pass
    [gate] 人工评审 -> commit

> /impl   # 产品实现, 消费 TD-001
IMPL-001 (τ_I: 产品实现)
    输出单元: ActivityTrendQuery.java (F2F 单文件)
    依赖语句: TD-001
    验收清单: 契约一致; 静态检查通过; 趋势正确性用例
    [validate] 静态检查通过; 与 TD-001 契约一致 -> pass
    [gate] 人工评审 -> commit -> 拼装进项目
```

三个要点值得强调。其一，历史即设计日志：语句序列完整记录了“为什么是这个设计”——每条决策都有语句可查，这是对话式工作流无法提供的可审计性。其二，回滚是局部的：评审不通过时，重生成的是一条语句，而不是一场对话的上下文。其三，控制台不隐藏模型：填槽与评审之间，模型仍然负责生成文档与代码——SpS 改变的是指令的形态，不是取消生成。

4.4 语句分解与拼装：与 F2F 的衔接

复杂 SQL 任务通过拆解为多条语句完成；同理，复杂项目通过拆解为多条规范语句完成。分解的单位与拼装的单位需要精确对应，否则语句产物会在集成时重新纠缠。这正是 F2F[9] 的位置：F2F 以特征为最小自治单元，单文件承载从入口、业务处理到数据访问的完整链路，文件间零耦合，维护采用整体替换。

SpS 与 F2F 的衔接是双向咬合的：

输出单元对齐：实现语句 τ_I 的输出单元即 F2F 特征文件。一条实现语句生成一个自治文件，文件的可替换性使“评审不通过 → 重生成整条语句”成为零副作用的操作——替换式维护在输入端有了对应物：语句重生成。

拼装顺序无关：F2F 的零耦合保证多个已提交的特征文件可以按任意顺序拼装进项目，语句的完成顺序不约束集成顺序；接口契约在校验层闭合（机制二），集成期不需要新的协商。

修改半径双端有界：输入端，缺陷至多存活到下一个阶段门禁（5.2 节）；输出端，缺陷的修复半径至多是一个特征文件。两端共同把“一次错误的代价”从不可预知压到有界。

这一衔接自然引出一个生命周期问题：项目建成之后，SpS 与既有 AI 编程工具如何分工？第 6 节给出形式化的回答——两个相位、一个迁移判据与一套交接协议。

4.5 SQL 类比的系统映射

表 3 给出两个体系的完整映射。

表 3: SQL 世界与 SpS 世界的系统映射

SQL 世界	**SpS 世界**
SQL 语法	通用规范 (T, G, V)
一条查询语句	一条规范语句 $s = (\tau, \delta)$
字面量 / 绑定变量	开发者细节 δ（槽位赋值）
数据字典 / schema	项目常量层 G
语法与类型检查	轻量校验（生成前）
查询计划	技术方案（τ_D 产物）
EXPLAIN 后执行	门禁评审后生成
事务 COMMIT / ROLLBACK	阶段提交 commit / 语句重生成 regenerate
结果集	阶段产物（文档 / 特征文件）
复杂任务拆为多条 SQL	复杂项目分解为多条语句
多结果集的应用层拼装	独立特征文件按契约拼装（F2F 单元）
查询控制台 / workbench	规范控制台
方言（MySQL / PostgreSQL）	规范方言（项目族模板变体）

映射在两处刻意中止，这同样有信息量。其一，SQL 有优化器在语法与执行之间自动选择路径，SpS 在技术方案阶段之后设有人工门禁——因为“软件的结构”不像访问路径那样有单一最优解，它携带业务判断，不应被自动化吞掉。其二，SQL 语句之间通常独立，规范语句之间存在消费关系（τ_D 消费 τ_P 产物）——这使 SpS 更像带依赖的批处理语言，也正因如此需要门禁而非仅仅需要解析器。

5 性质与讨论

本章陈述范式的三条核心性质，划定其与自由提示词工作流的边界，并诚实列出成本与局限。

5.1 性质一：注意力集中性

命题 1 在两端各有可操作的含义。模型端：结构决策的方差归零后，上下文中不再出现“自相矛盾的结构指令”，稀释源被移除，细节召回的改善超出模板文本本身的长度收益 [8]。人端：评审对象从自由文本变为槽位赋值表，评审动作退化为核对——“这些细节是否是我要的”，而非“这份产物整体是否可信”。前者是抽样，后者是通读；注意力消耗相差一个量级。

需要澄清的是：注意力集中性不承诺总注意力下降（模板本身占据固定开销），它承诺的是注意力的方差与错配下降——注意力从“重新决定已知的事”转移到“决定未知的事”。这正是把注意力作为关键变数的含义：范式优化的目标是注意力分配的质量，而非生成产物的数量。

5.2 性质二：错误早绑定与传播有界

命题 2（传播有界）在阶段门禁流水线 $\Pi = (P \to D \to I)$ 中，第 k 阶段引入的缺陷至多存活到第 k 阶段的出口门禁；修复动作是重生成该阶段语句，不触碰已提交的上游产物。若缺陷逃逸到第 $k+1$ 阶段，其修复需要跨阶段回溯，期望成本由修改半径与重审半径之积决定。

Boehm 曲线 [17] 在此获得一个新的放大因子：LLM 实现错误设计与实现正确设计的成本近乎相同 [2, 3]，因此“晚发现的错误”不再以工时计价，而以“已高速生成的错误产物体积”计价。门禁的价值不在评审本身，而在给错误一个尽早显形的固定位置。配合输出端的 F2F 单元（4.4 节及 [9]），即使缺陷逃逸到实现阶段，修复半径仍被压缩到单文件替换。

5.3 性质三：评审友好性

命题 3（评审有界）自由智能体会话的评审对象是跨文件、大小不可预知的 diff；SpS 的评审对象由模板结构保证有界：阶段 k 的评审 = 一份模板化文档（τ_P、τ_D 产物）或一个特征文件（τ_I 产物）加一组槽位级差异。

这一性质直接回应 GitClear 度量揭示的失控模式 [4]：无结构生成 + 弱评审导致重复率上升与熵增。评审友好性使“每阶段人工 review”从理想主义口号变成可持续的日常动作——评审带宽是整个范式的吞吐上限，命题 3 保证该上限不被产物体积击穿。

5.4 与自由提示词工作流的边界

SpS 不是自由提示词的替代，而是其上的一个结构化层。以下场景自由提示词仍然占优：探索式原型（结构本身是待发现物，模板会过早锁死搜索空间）；单文件小修（三阶段流水线的固定开销超过收益）；创意性生成（写作、设计草稿，无“结构正确性”可言）。判断准则与命题 1 一致：任务中不变量占比越高、阶段链越长、错误传播代价越大，SpS 的收益越大。全新项目的正向构建——本文的靶心场景——三项指标同时取最大值。需要说明的是，本节判据回答的是任务维度（哪些任务适合 SpS）；项目时间维度的分工——建成之后如何回落、何时重入——由第 6 节的生命周期分工模型回答。

5.5 成本与局限

模板僵化与规范锁定。模板在消除结构方差的同时也会消除结构多样性；模板盲区内的好设计会被模板排除。缓解：模板版本化，方言允许项目族变体（表 3），且锁定只作用于正向构建期——微调期不受约束（第 6 节）。

小任务开销。三阶段流水线对琐碎任务是负收益。缓解：5.4 节的判断准则；控制台允许语句级跳过（单阶段语句）。

规范自身的治理。通用规范是需要演化的人工制品：模板改版、常量层升级、槽位增删都会引入迁移问题，如同 schema 迁移。这是被 SQL 类比照亮的一个成本：语法不是免费的，但语法的收益随语句数量摊薄。

门禁的等待成本。阶段门禁引入人工等待。缓解：门禁异步化（提交后继续草拟下一条语句，评审通过前不生成下游产物），以及评审本身的槽位化（5.1 节）把单次评审成本压低。

对生成模型的不可见性。SpS 在交互界面层运作，不改变模型权重；模板带来的收益依赖模型对模板的遵循度。低能力模型可能连模板一致性都无法维持——但这恰恰说明 SpS 与模型能力是互补而非替代关系：模型越强，语法释放的注意力越多。

6 生命周期分工模型

SpS 与传统 AI 编程工具的关系不是替代，而是生命周期上的分工。本章将这一分工形式化：以结构决策熵（定义 3）随时间的演化为参量，项目生命周期划分为两个相位，各有其占优工具；相位之间由一个可操作的迁移判据与一套交接协议连接。

6.1 两个相位

相位一：结构生成期（0→1）。项目从零推进到“结构完整、功能可用”。此相位的特征是结构决策熵处于峰值：架构、组织、风格等每一个结构决策都未定，不变量尚未沉淀，任何缺陷的传播半径都由尚未成形的结构决定。这是 SpS 的作用域：通用规范吸收不变量（命题 1），阶段门禁有界化错误传播（命题 2），语句分解配合 F2F 单元完成拼装（4.4 节）。5.4 节的三项判据——不变量占比、阶段链长度、错误传播代价——在此相位同时取最大值。

相位二：局部微调期（1→1.1）。功能可用之后的演化：缺陷修复、局部增强、参数与文案调整。此相位的特征恰好反转：结构已冻结为 F2F 特征文件，单次修改的修改半径有界——至多一个文件，整体替换 [9]；修改不引入新的结构决策，熵增量近似为零。此时三阶段流水线的固定开销（模板填充、双重校验、门禁等待）超过其方差消除收益，传统 AI 工具（对话式、补丁式）反而占优：它们以文件为上下文单元的会话形态恰好匹配“修改半径 ≤ 1”的任务；F2F 的特征文件为这类工具提供了安全的修改单元——一次微调至多影响一个文件，文件即边界。

6.2 相位迁移判据

两个相位之间的迁移由一个可操作的判据给出。

判据（相位迁移）。对下一次修改 m，设 $\Delta H(S \mid m)$ 为其引入的新增结构决策熵，$r(m)$ 为其预期修改半径。当 $\Delta H(S \mid m) \approx 0$ 且 $r(m) \leq 1$ 个 F2F 单元时，m 属于局部微调，应交由传

统工具在文件级上下文中完成；当 $\Delta H(S \mid m)$ 显著为正（新增跨文件特征、架构调整、技术栈变更）时，m 是一次结构性变更，应重入 SpS 会话，重新走语句与评审。

判据把“项目建成后如何继续”从口味问题变成可检验的边界问题，并预示生命周期的循环形态：微调期的持续累积（$1 \to 1.1 \to \ldots$）最终触发结构性变更，重入新一轮语句化构建（$0' \to 1'$）。两个相位的系统对比见表 4。

表 4: 两个相位的系统对比

维度	**相位一：结构生成期（0→1）**	**相位二：局部微调期（1→1.1）**
结构决策熵	峰值，$H(S \mid p) > 0$	熵增量 $\Delta H(S \mid m) \approx 0$
修改半径	未定，由结构形态决定	有界，$r(m) \leq 1$ 个 F2F 单元
占优工具	SpS（规范控制台）	传统 AI 工具（对话式、补丁式）
工作单元	规范语句 $s = (\tau, \delta)$	特征文件（文件即边界）
核心机制	语法吸收不变量 + 阶段门禁	文件级上下文 + 整体替换
相位迁移	commit 阶段产物后进入相位二	结构性变更（$\Delta H > 0$）时重入相位一

6.3 交接协议

分工不是断崖：两相之间有明确的交接资产，SpS 会话提交的产物在微调期继续服役。

规范即上下文。τ_D 产物（架构风格、接口契约、异常策略）是微调期最精确的 grounding 材料——传统工具的会话以文件为上下文，而规范文档本身就是文件，可直接引用。相比从代码反推设计，规范把“为什么是这样”显式携带进微调期。

常量层即约束。项目常量层 G（技术栈、命名、风格约定）在微调期充当轻量护栏：即使是对话式工具，也可将 G 注入会话上下文，抑制微调漂移。

历史即审计。语句序列（设计日志）为微调提供决策考古层：修改一个特征文件前，可查其生成语句与上游契约（4.4 节的依赖关系），避免微调破坏既有契约。

6.4 与“取代论”的分别

生命周期分工是本文与既有主张的一个明确分别：SpS 不宣称自己是更好的 Cursor 或 Claude Code，也不宣称微调期应继续留在语句界面里——正相反，它把“何时离开”写进了范式本身。这与 SQL 世界的形态一致：没有人用查询控制台写数据清洗的一次性脚本，也没有人因此否认 SQL。工具的作用域声明是语法成熟的标志，而非弱点。

7 扩展讨论：开放问题

本节给出三个超出本文论证范围、但决定范式命运的问题。

规范即数据。语句序列是结构化、可 diff、可统计的记录。这使开发过程本身第一次成为可分析的数据集：结构决策的演化史、缺陷被捕获的门禁位置分布、槽位填充与返工率的相关性——注意力经济学的命题由此从论证变为可度量。一个自然的度量是注意力集中比 κ：评审与生成中分配给变量 δ 的注意力份额；SpS 的实证议程即验证 κ 与交付质量的正相关。与之配套的是定义 3 的操作化：结构决策熵可经模板覆盖率近似测量——一次会话中由模板与常量

层确定的结构决策占比越高，$H(S \mid \tau) \to 0$ 的承诺兑现得越充分；生命周期分工（6.2 节）的 $\Delta H(S \mid m)$ 同样可循此路径估计。

方言与生态。SQL 的方言分裂提示了通用规范的可能命运：不同项目族、不同组织的模板变体终将分化。分化未必是坏事——方言竞争是语法演化的机制；但跨方言的语句迁移成本值得预研。

与 F2F 的合流。SpS 组织输入端、F2F 组织输出端，两者共享同一经济学根基：人类时代的效率产物（自由提示词、水平分层）在生成成本趋零的时代失效。合流后的完整方法论可以概括为一句话：输入声明式，输出自治式——人写的每条指令是一条有语法的语句，机器产出的每个单元是一个有边界的文件。

8 结论

本文的工作可以概括为一句话：把 AI 编程的输入端从自由文本重构为有语法的语句，并界定这套语法在项目生命周期中的作用域。

本文从注意力经济学的视角，提出面向 AI 编程交互界面的重构范式 Spec-per-Stage：通用规范充当 SQL 语法，规范语句表达任务，轻量校验前移廉价失败，阶段门禁有界化错误传播；输入端与 F2F 的输出端范式衔接，并以生命周期分工模型界定了其与传统 AI 编程工具的作用域边界，共同构成面向 AI 编程时代的完整方法论。

对 AI 编程的失控症状 [4]，本文给出的诊断不是模型不够强，而是界面组织不当：自由提示词把稀缺的注意力拍卖给了不变量。修复的动作与关系模型当年相同——声明式化。SQL 把“怎么取”交给优化器，把注意力还给“要什么”；SpS 把“怎么建”交给语法与模型，把注意力还给“建什么”。

AI 编程的下一个杠杆，可能不在更强的模型，而在更好的语法。把不变量吸收进规范，把注意力还给注意力该在的地方。

参考文献

[1] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer. The Impact of AI on Developer Productivity: Evidence from GitHub Copilot, 2023. [EB/OL]. arXiv:2302.06590.

[2] Mark Chen, Jerry Tworek, Heewoo Jun, et al. Evaluating Large Language Models Trained on Code, 2021. [EB/OL]. arXiv:2107.03374.

[3] Angela Fan, Binyamin Gokkaya, Mark Harman, et al. Large Language Models for Software Engineering: Survey and Open Problems. *ACM Transactions on Software Engineering and Methodology*, 33(8):1–79, 2023.

[4] GitClear. Coding on Copilot: 2023 Data Suggests Downward Pressure on Code Quality. 报告, GitClear, Seattle, 2024. [R].

[5] E. F. Codd. A Relational Model of Data for Large Shared Data Banks. *Communications of the ACM*, 13(6):377–387, 1970.

[6] Donald D. Chamberlin and Raymond F. Boyce. SEQUEL: A Structured English Query Language. In *Proceedings of the 1974 ACM SIGFIDET Workshop on Data Description, Access and Control*, pages 249–264, 1974.

[7] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. Attention Is All You Need. In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017.

[8] Nelson F. Liu, Kevin Lin, John Hewitt, et al. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

[9] Zikun Yang. Feature-per-File: Refactoring Code Organization for AI Programming — A Data-Engineering Perspective, 2026. Preprint.

[10] David Lorge Parnas. On the Criteria To Be Used in Decomposing Systems into Modules. *Communications of the ACM*, 15(12):1053–1058, 1972.

[11] ISO/IEC/IEEE. ISO/IEC/IEEE 29148:2018, Systems and software engineering — Life cycle processes — Requirements engineering, 2018. [S].

[12] GitHub. Spec Kit: Toolkit to Use Spec-Driven Development. https://github.com/github/spec-kit, 2025. [EB/OL]. [2026-09-17].

[13] AWS. Kiro: The AI-Powered IDE. https://kiro.dev, 2025. [EB/OL]. [2026-09-17].

[14] Tao Yu, Rui Zhang, Kai Yang, et al. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018.

[15] Pengfei Liu, Weizhe Yuan, Jinlan Fu, et al. Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing. *ACM Computing Surveys*, 55(9):1–35, 2023.

[16] Winston W. Royce. Managing the Development of Large Software Systems. In *Proceedings of IEEE WESCON*, pages 1–9, 1970.

[17] Barry W. Boehm. *Software Engineering Economics*. Prentice-Hall, Englewood Cliffs, 1981.

[18] Peter Naur. Programming as Theory Building. *Microprocessing and Microprogramming*, 15(5):253–261, 1985.

[19] Shunyu Yao, Jeffrey Zhao, Dian Yu, et al. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*, 2023.

[20] John Ousterhout. *A Philosophy of Software Design*. Yaknyam Press, Palo Alto, 2018.